

tp css partie 2

Matisse

kra

bts sio 1

Sommaire

Problème	2
mise en place de la solution	2
Sous problème 1 l'ordre de priorité	2
Sous problème 2 Display	3
1) Display:block	3
2) Display: line	4
Sous problème 3 menu déroulant:	7
Sous problème 7 display flex	8
sous problème 8 justify-content	10
sous problème 10 Flexbox	11
Sous probleme 11 mise en page global	12
conclusion	13
annexe	13

Problème

ce tp est la suite du TP html/css l'objectif est de tester et approfondir ses connaissances dans ce domaine

travail effectué sur mac dans le logiciel vs code

mise en place de la solution

le document TD3 CSS : HTML/CSS Avancé (2/2) se présente comme un guide sur lequel je me suis appuyé pour décomposer la solution en plusieurs sous problèmes.

Sous problème 1 l'ordre de priorité

comme vu dans les précédents tp les sélecteurs servent à sélectionner un ensemble de balises sur lesquels on applique une règle CSS. Nous avons appris lors du TD1 les sélecteurs de base et lors du TD2 la combinaison de sélecteurs.

Plusieurs règles CSS peuvent porter sur un même élément HTML. Si ces règles peuvent coexister, elles sont toutes appliquées. Par exemple, si vous avez le code CSS suivant :

```
div { background-color:blue; }  
.skill { color:red; }
```

alors un `<div class="skill">` aura les deux propriétés `background-color:blue` et `color:red`.

Mais il peut aussi arriver que ces dernières soient contradictoires. Par exemple, si on a le code CSS suivant, de quelle couleur sera le texte d'un `<div>` de classe skill ?

```
div { color:blue; }  
.skill { color:red; }
```

Pour régler ces conflits, le CSS définit une notion de priorité. Pour savoir comment sont établis il faut connaître les différentes façons de déclarer du styles CSS :

1. style interne
utiliser un fichier de style externe et le lier au document HTML avec la balise `<link>` dans le `<head>`
2. style externe: inclure des règles CSS directement dans le `<head>` à l'aide de la balise `<style>`
3. style inline: inclure des règles CSS directement dans une balise à l'aide de l'attribut `<style>`

La priorité d'un sélecteur CSS est une valeur (a,b,c,d) définie comme ceci :

- la valeur "a" code la priorité basée sur l'emplacement de la règle. Par ordre de priorité décroissante, nous avons
 - a=2 pour les styles inline,
 - a=1 pour les styles externes et internes,
 - a=0 pour le style par défaut du navigateur.

Si a est strictement plus grand alors la règle CSS est plus prioritaire. En cas d'égalité,

- on regarde la valeur "b" compte le nombre de sélecteurs d'identifiants (e.g. #id), Si b est strictement plus grand (et donc a égal) alors la règle CSS est plus prioritaire. En cas d'égalité sur a et b,
- on regarde "c" compte le nombre de sélecteurs de classes (e.g. skill) ou pseudo-classes (:hover, visited,...) En cas d'égalité, on regarde la "quatrième lettre" d.

- d compte le nombre de sélecteurs de balises (e.g. div, span) ou de pseudo-éléments (e.g. :first-letter, after)
- les opérateurs de combinaison et le sélecteur universel * ne contribuent pas à la priorité.
- en cas d'égalité absolue, la règle écrite en dernier est prioritaire.

voici des exemple de comment sont calculé les priorité :

Dans ces exemples, toutes les règles sont dans un fichier externe donc a=1.

.titi span

1011 a=1 b=0 (pas d'#id) c=1 (1 sélecteur) d=1(1 sélecteur balise)

Div span

1002

Nav.titi.tata div div div div div

1025

ul li div.skill

1011

#id

1100

Div > a

1001

Div + a

1001

autre exemple

1)

<html>

<head>

<style type="text/css"> p {color: pink} </style>

<link rel="stylesheet" type="text/css" href="css/styles.css"> </head>

<body><p style="color:red">Priorité CSS</p></body> </html>

ici la couleur PrioritéCSS sera en rouge car le style est inclu en inline et je le rappelle a=2 pour les style inline

2)

<html>

<head>

<style type="text/css"> p {color: pink} </ style>

<link rel="stylesheet" type="text/css" href="css/styles.css"> </head>

<body><p>Priorité CSS</p></body> </html>

ici la couleur aura la couleur du style extérieur car on est dans le cas ou il y a égalité absolue et donc c'est la dernière règle qui est prise en compte le style externe est inclu après le style interne donc c'est dans ce style que y a la dernière regle

Sous problème 2 Display

Comme nous l'avons vu précédemment une variable admet 4 boite,content padding border margin.c'est boite sont gérés par l'élément Display. nous allons voir trois valeur de Display

1) Display:block

Les éléments block sont des éléments :

- qui par défaut occupent toute la largeur,
- qui provoquent un saut de ligne avant et après son affichage (que l'on ait diminué sa largeur ou pas)
- dont on peut définir la taille en CSS via les propriétés height et width.

! Notez que les balises de structure que l'on a présentées au TD précédent ont display:block comme style par défaut dans le navigateur.

2) Display: line

Les éléments inline sont des éléments :

- qui prennent leur taille en fonction de leur contenu,
- qui ne provoquent pas un saut de ligne, ils sont écrits comme du texte les uns à la suite des autres
- dont on ne peut pas définir la taille en css via les propriétés height et width,

j'ai tester le mode block dans la page chuck norris du précédent tp j'ai modifier le menu de mon site qui était une liste puisque <nav> est display block mes différents lien du menu s'empile par default

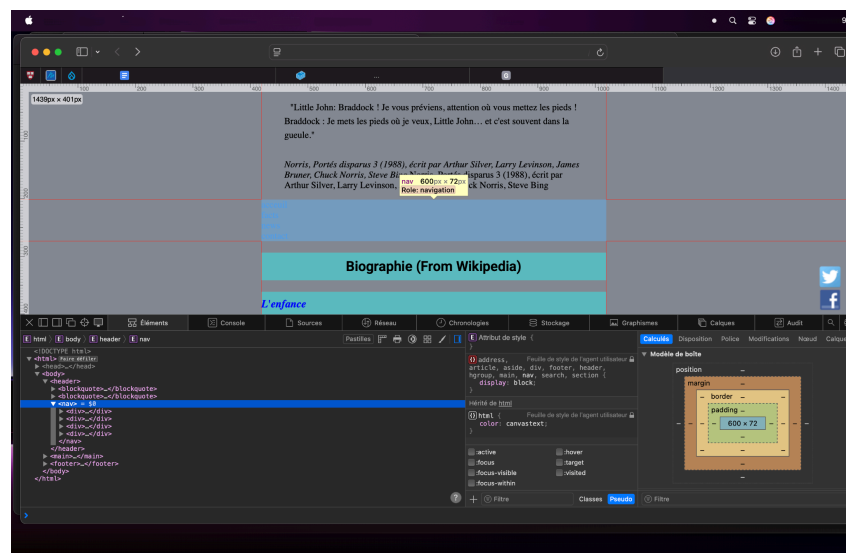
```
<nav>
  <ul>
    <li> <a href="index.html"> accueil</a></li>
    <li><a href="contact.html">contact</a></li>
  </ul>
</nav>
```

avant

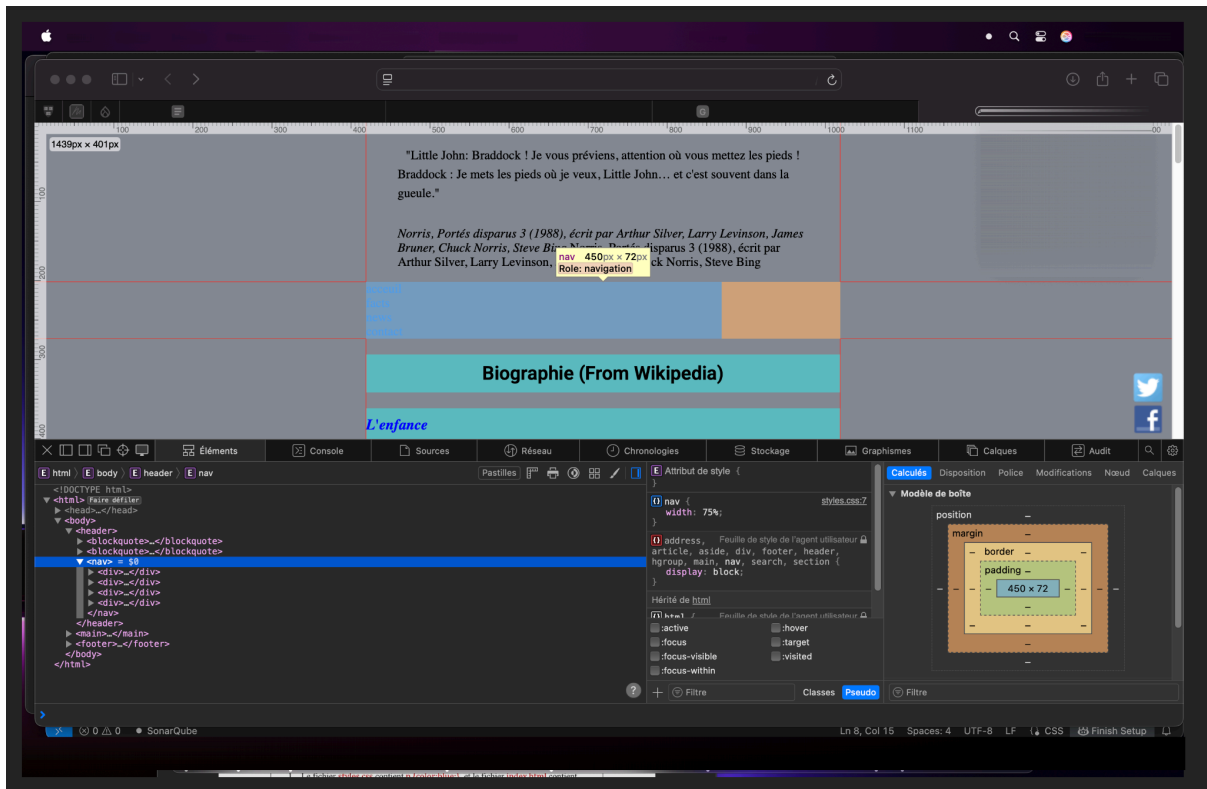
```
<nav>
  <div> <a href="index.html"> accueil</a></div>
  <div> <a href="facts.html"> facts</a></div>
  <div> <a href="news.html"> news</a></div>
  <div><a href="contact.html">contact</a></div>
</nav>
```

après

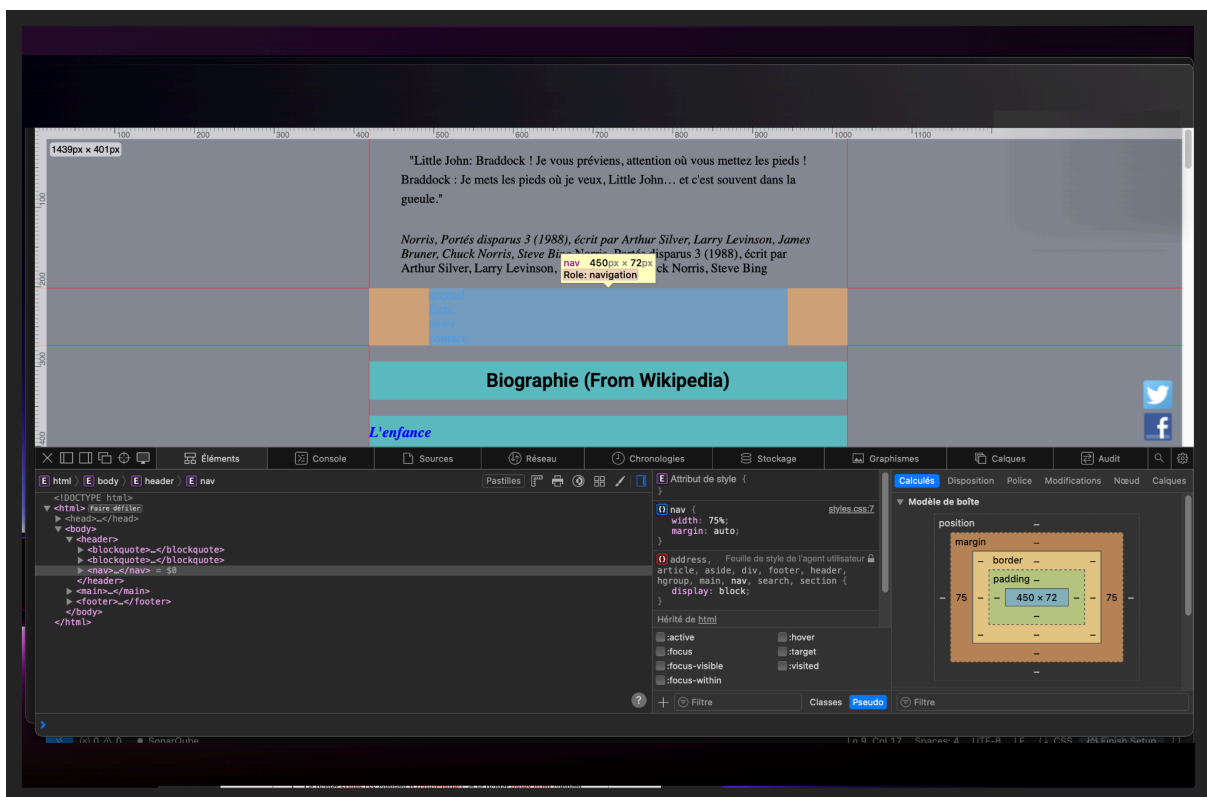
Puisque <nav> est display:block il devrait prendre toute la largeur de la page <le body> comme expliqué précédemment mais ce n'est pas le cas le nav ne prend en faite que la largeur de son blockparent le plus proche qui est dans ce cas le <header>



puisque nav est de type block je peux lui fixer des dimension en lui attribuant une largeur de 75% on peut apercevoir que sa largeur a bien diminué de 25% la partie rouge est la largeur de base lorsque je fais inspecter sur la page



en utilisant la commande `margin: auto;` dans mon code css j'ai centré horizontalement mon nav dans le header



ensuite pour plus de personnalisation j'ai ajouté une règle dans mon css pour modifier la couleur de fond de mes div du menu j'ai utilisé cette commande

```
div{
  background-color: #5BBDBF;
}
```

accueil
facts
news
contact

avant

accueil
facts
news
contact

après

puis j'ai créé une nouvelle règles afin de modifier la couleur de fond de mes lien juste pour le menu j'ai utiliser cette commande:

```
div a{
  background-color: #c0d5c2;
}
```

div sélectionne tous les div de mon html et le **a** qui est après permet de sélectionner tous les a qui sont dans les div sans sélectionner les a a l'extérieur des div

accueil
facts
news
contact

A présent j'ai tester le mode inline dans les <div> fils de <nav> je leur attribuer le display inline par conséquent les lien ne sont plus empilés mais tous sur une ligne ensuite pour supprimer les espace ou entrée du menu j'ai utiliser des commentaires pour les supprimer puis dans mon css j'ai ajouter une hauteur de 50px a mon **nav** le **nav** est block donc on peut lui donner une hauteur, du padding horizontal sur mes <a> ainsi qu'une bordure

```
div{
  display: inline;

  background-color: #5BBDBF;
}
div a{
  padding:0px 10px ;
```

```
border-left: 2px solid black;
background-color: #c0d5c2;
}
nav{
  height: 50px;
  width: 75%;
  margin: auto;
}
```

accueil | facts | news | contact

Sous problème 3 menu déroulant:

j'ai ajouté des sous menu pour la rubrique accueil et contact.

- j'ai ajouter mes sous menu a la balise navigation après les balise <a> contenant le lien accueil et le lien contact afin que mes sous menu on pour père le même <div> mes div étant display inline mes sous menu ce sont juste rajouter a la suite des menu "accueil facts news et contact"

```
<nav>
  <div> <a href="/index.html"> accueil</a>
    <div class="submenu">
      <div><a href="/one.html">one</a></div>
      <div><a href="/two.html">two</a></div>
      <div><a href="/three.html">three</a></div>
    </div>
  </div>
  <div> <a href="facts.html"> facts</a></div>
  <div> <a href="news.html"> news</a></div>
  <div><a href="contact.html">contact</a>
    <div class="submenu">
      <div><a href="/other.html">other</a></div>
      <div><a href="/another.html">another</a></div>
    </div>
  </div>
</nav>
```

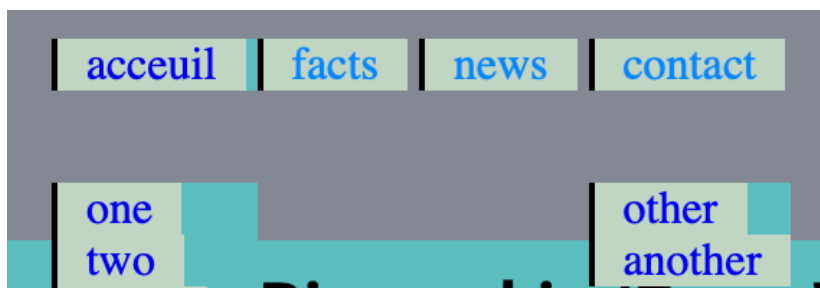
accueil | one | two | three | facts | news | contact
other | another

- ensuite pour bien positionner mes sous menu j'ai fait en sorte que l'affichage du reste de la page se positionne comme si ces sous menu n'existait pas.j'ai cree une regles css pour mes menu en utilisant le selecteur submenu puis à mes sous menu j'ai ajouter un décalage de 50px par rapport au haut et un décalage de 0px par

rapport à la gauche a ce moment la mes sous menu sont mal placés car il se sont placé par rapport à la balise <body> or je souhaite qu'il se place par rapport à leur div parent j'ai donc créé une nouvelle règle css avec la position relative la position relative va faire en sorte que mon menu soit positionné

-

```
.submenu{
    position: absolute;
    top: 50px;
    left: 0px;
}
nav div{
    position: relative;
}
```



- maintenant que mes sous menu son positionner au bonne endroit j'ai utilisé display:None; a mes règle des sous menu pour les cacher de page et puis j'ai créé une règle qui vérifie si les père <div> des sous menu sont survolé puis j'ai inclus dans cette règle display block pour les afficher

- ```
nav div:hover .submenu{display: block;}
```

### Sous problème 7 display flex

A ce niveau les sous menu apparaissent bien lorsque l'on survole accueil et contact mais on ne peut pas y accéder on va donc utiliser display flex pour améliorer l'apparence de notre menu et puis résoudre ce problème.

- dans la règle css de nav on donne l'élément `display: flex;` a présent si dans les div on change display inline en block rien ne se passe mais si on attribue la déclaration `flex-direction: column;` le menu s'arrange comme un display block



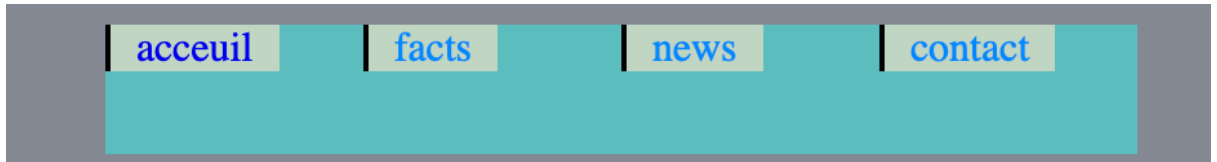


display: flex;

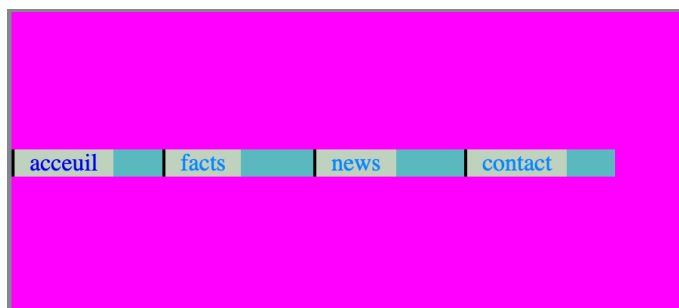


flex-direction: column;

- j'ai donner une largeur de 100px au <div> descendant de <nav> pour que les menu et les sous menu aient la meme largeur



- je veux que les <div> titre des menu soit centrer verticalement en prenant toute la hauteur pour mieux comprendre le comportement de display flex :
  - j'ai donné au <nav> une couleur de fond violette d'une hauteur de 200px et au sous menu un décalage de 200px vers le bas. puis j'ai utiliser dans les <nav> la propriété align-items ce qui a centrer verticalement les titre des menu par rapport au nav.
  - avec cette propriété la hauteur des div est raccourcis



```
.submenu{
 display: none;
 position: absolute;
 top: 50px;
 left: 0px;
 bottom: 200px;
 background-color: #aca;
}
```

```
nav{
 height: 200px;
 align-items: center;
 width: 75%;
 margin: auto;
 display: flex;
 flex-direction: row;
 background-color: #FF00FF;
}
```

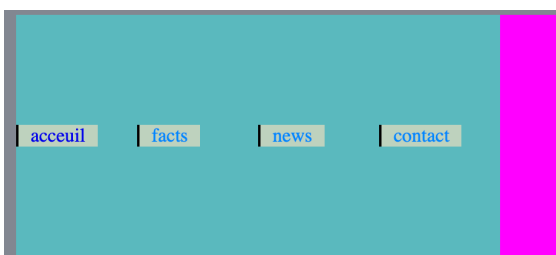
- si on utilise le stretch a la place de center on peut voir que les <div> prennent toute la hauteur du <nav> mais ils ne sont plus centrés verticalement et les menus sont de nouveau accessibles.



```
nav{
 height: 200px;
 align-items: stretch;
 width: 75%;
 margin: auto;
 display: flex;
 flex-direction: row;
 background-color: #FF00FF;
}
```

- à l'aide des deux propriétés j'ai fait en sorte que mes <div> prennent toute la hauteur du <nav> et que mes liens <a> soient centrés verticalement dans mes <div>. Pour ce faire dans mon <nav> j'ai utilisé `align-items: stretch;` résultat mes div ont pris toute la hauteur ensuite dans mon j'ai utilisé `display flex` suivi de la propriété `align-items: center;` ce qui a centré verticalement mes liens <a>.

```
nav > div{
 display: flex;
 width: 100px;
 background-color: #5BBDBF;
 position: relative;
 align-items: center;
}
```



### sous problème 8 justify-content

je devais justifier mes blocs titre des menus pour les placer à droite <nav> j'ai utilisé la propriété `justify-content` pour les justifier suivi de l'instruction `flex-end` ce qui permet de les placer à la fin du nav donc à droite

```

nav > div{
 display: flex;
 justify-content: flex-end;
 width: 100px;
 background-color: #5BBDBF;
 position: relative;
 align-items: center;
}

```



avant



après

### sous problème 10 Flexbox

je devais manipuler le <header> pour aligner la citation et mon menu sur la même ligne, tout en plaçant ce menu en bas du header.

- Pour cela, j'ai transformé le <header> en conteneur flex grâce à la propriété display: flex;. J'ai ensuite utilisé : justify-content: space-between; pour placer la citation à gauche et le menu à droite, align-items: flex-end; pour positionner verticalement le menu en bas du header, et j'ai ajouté un fond blanc pour vérifier le positionnement visuel. Cette approche avec Flexbox permet d'obtenir un alignement propre et réactif, sans utiliser de positionnement absolu ni de marges forcées..

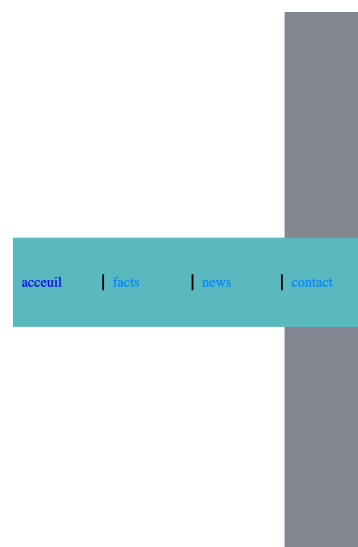
```

header {
 display: flex;
 flex-direction: row;
 justify-content: space-between;
 align-items: flex-end;
 background-color: white;
 padding: 10px;
}

```

"Little  
John:  
Braddock  
! Je vous  
préviens,  
attention  
où vous  
mettez  
les pieds  
!  
Braddock  
: Je mets  
les pieds  
où je  
veux,  
Little  
John... et  
c'est  
souvent  
dans la  
gueule."

Norris,  
Portés  
disparus  
3 (1988),  
écrit par  
Arthur  
Silver,  
Larry  
Levinson,  
James  
Bruner,  
Chuck  
Norris,  
Steve  
Bing  
Norris,  
Portés  
disparus  
3 (1988),  
écrit par  
Arthur  
Silver,  
Larry  
Levinson,  
James  
Bruner,  
Chuck  
Norris,  
Steve  
Bing



## Sous problème 11 mise en page global

Dans cet exercice, il était demandé d'organiser le contenu principal du site avec Flexbox, en respectant des proportions précises entre les différentes zones de la page.

Dans un premier temps, j'ai fixé la largeur du `<body>` à 900 px et centré la page à l'intérieur de la fenêtre grâce à `margin: auto;`.

Dans un deuxième temps, j'ai appliqué `display: flex;` sur la balise `<main>` pour afficher ses enfants côte à côte comme deux colonnes.

Pour respecter les consignes :

- `<article>` fait 60 % de large,
- `<aside>` fait 30 % de large + une marge gauche de 10 % sur son parent, afin de séparer visuellement les deux blocs,
- et pour bien marquer leur zone visuelle, j'ai donné un fond gris clair (`#CCC`) à ces éléments.

Ainsi, on obtient une mise en page équilibrée, simple et claire qui exploite parfaitement la flexibilité procurée par le modèle de mise en page Flexbox.

```
article {
 width: 60%;
 background-color: #CCC;
 padding: 10px;
}

aside {
 width: 30%;
 margin-left: 10%;
 background-color: #CCC;
 padding: 10px;
}
```

## Sous problème 12 cacher ou enlever un element du rendu

Cet exercice avait pour but de donner une certaine interaction au menu par l'apparition d'une petite puce à gauche du texte lors du survol d'un élément du menu.

Cette puce a été ajoutée dans chaque lien du menu par un élément `<span class="puce"></span>`, avec le style suivant :

- une forme circulaire (`border-radius: 50%`) ;
- une taille réduite (`width` et `height` de 6 px) ;
- une visibilité non-visible par défaut (`visibility: hidden`).

Par la règle suivante, la puce est rendue visible à survol du parent :

```
nav .submenu:hover .puce {
 visibility: visible;
}
```

Etant donné que la puce réserve de l'espace même si elle est invisible, on évite le tremblement du texte lors d'un survol.

Je retire aussi de couleur de fond aux le lien <a> pour renforcer leur lisibilité pour le menu.

## conclusion

Ce travail pratique m'a permis d'utiliser différentes techniques habituelles de « layout » avec Flexbox et de me familiariser avec le positionnement des éléments en CSS.

J'ai appris à organiser avec soin (header, main, aside, footer) le contenu d'une page, à gérer ses alignements, que ce soit horizontalement ou verticalement, à produire des effets d'interactions simples comme l'apparition d'une puce au survol d'un menu.

Ces mises au point m'ont permis d'apprendre à mieux appréhender la logique des modèles flex tout en ayant une page propre, équilibrée et fonctionnelle.

## annexe

```
/* ----- STYLES GÉNÉRAUX ----- */
body {
 background-color: #838892;
 width: 900px; /* largeur demandée */
 margin: auto; /* centre le body dans la fenêtre */
}
main {
 display: flex; /* transforme main en conteneur flex */
 flex-direction: row; /* met les enfants côte à côte */
 justify-content: space-between; /* laisse de l'espace si nécessaire */
}
article {
 width: 60%;
 background-color: #CCC; /* fond gris clair */
 padding: 10px;
}
aside {
 width: 30%;
 margin-left: 10%;
 background-color: #CCC; /* même fond que article */
 padding: 10px;
}
h2 {
 text-align: center;
 font-family: "Roboto";
 font-weight: 600;
}
```

```

}

h2, h3 {
 padding: 10px 0;
}

p {
 line-height: 150%;
 text-indent: 10px;
 margin: 30px 0;
}

h3 {
 font-style: italic;
 color: blue;
 border: 1px #CCCCCC;
}

h2.pair, h3.pair {
 background-color: #5BBDBF;
}

h2.impair, h3.impair {
 background-color: #FF5850;
}

.Spécialiste {
 clear: left;
}

.skill {
 font-style: italic;
 color: navy;
}

a {
 text-decoration: none;
}

a:visited {
 color: #0088FF;
}

a:hover {
 color: orange;
}

```

```

}

thead {
 background-color: #00AAFF;
}

th .skill {
 color: #640051;
}

table tbody:nth-of-type(even) tr {
 background-color: #ccc;
}

table tbody:nth-of-type(odd) tr {
 background-color: white;
}

.chuck {
 color: goldenrod;
}

table.skill {
 padding: 0 5px;
}

.jeune {
 float: left;
}

.beware {
 height: 300px;
 float: right;
}

/* ----- ICÔNES DE RÉSEAUX ----- */
.reseaux {
 position: fixed;
 bottom: 40px;
 right: 10px;
 width: 40px;
 margin-bottom: 5px;
}

.reseau {

```

```

 position: fixed;
 bottom: 0;
 right: 10px;
 width: 40px;
 margin-bottom: 5px;
}

/* ----- MENU PRINCIPAL ----- */
nav {
 width: 75%;
 margin: right;
 height: 50px;

 /* Pour vérifier le placement à droite */
 background-color: transparent;

 display: flex;
 flex-direction: row;
 justify-content: flex-end; /* aligne tout le menu à droite */
 align-items: stretch; /* les div prennent la hauteur complète */
}

nav > div {
 display: flex;
 width: 100px;
 background-color: #5BBDBF;
 position: relative; /* nécessaire pour positionner les sous-menus */
 align-items: center; /* centre verticalement le texte */
}

nav a {
 background-color: transparent;
 padding: 0 10px;
 border-left: 2px solid black;
 display: block; /* meilleure surface cliquable */
}

.puce {
 display: inline-block;
 width: 6px;
 height: 6px;
 border-radius: 50%; /* forme circulaire */
 background-color: black;
 visibility: hidden; /* invisible par défaut */
}

nav .submenu:hover .puce {

```



```

 visibility: visible;
}
/* Supprimer la bordure du premier lien */
nav > div:first-child a {
 border-left: none;
}

/* ----- SOUS-MENUS ----- */
.submenu {
 display: none; /* masqué par défaut */
 position: absolute;
 top: 60px; /* décalage vertical sous le menu */
 left: 0;
 background-color: #aca;
}

nav div:hover .submenu {
 display: block; /* visible au survol du parent */
}

header {
 display: flex; /* active flexbox */
 flex-direction: row; /* aligne la citation et le menu côte à côte */
 justify-content: space-between; /* espace entre citation (gauche) et menu
(droite) */
 align-items: flex-end; /* aligne le menu en bas du header */
 background-color: white; /* fond blanc demandé */
 padding: 10px;
}

/* ----- FIN DU MENU ----- */

```